

Secured WebShop



Thomas Peltier – CID2a
ETML vennes
24p
Gaël Sonney

Table des matières

1	SPÉCIFICATIONS.....	3
1.1	TITRE.....	3
1.2	DESCRIPTION.....	3
1.3	MATÉRIEL ET LOGICIELS À DISPOSITION	3
1.4	PRÉREQUIS	3
1.5	LISTE DES ACTIVITÉS OBLIGATOIRES	3
1.5.1	Implémenter une page de login en frontend	3
1.5.2	Implémenter une page d'inscription en frontend	4
1.5.3	Remplacer les mots de passes en clair dans la base par un hash	4
1.5.4	Ajouter un sel.....	5
1.5.5	Ajouter un poivre	5
1.5.6	Corriger les requêtes existantes afin de prévenir l'injection SQL.....	5
1.5.7	Implémenter l'utilisation d'un token JWT.....	5
1.5.8	Ajouter les rôles administrateur et utilisateur dans le JWT et protéger les routes d'administration	6
1.6	LISTE DES ACTIVITÉS « FACILES » À CHOIX	6
1.6.1	Mettre en place le HTTPS	6
1.6.2	Mettre en place une politique de mot de passe fort.....	6
1.6.3	Limiter la durée du token JWT actuel et implémenter un refresh token pour rester connecté sur une longue période	6
1.6.4	Effectuer un audit des dépendances NPM	7
1.6.5	Gérer les exceptions afin de ne pas retourner trop d'information en cas d'erreur	7
1.7	LISTE DES ACTIVITÉS « MOYENNES » À CHOIX	7
1.7.1	Limiter le nombre de tentatives de login (5 essais par minute par IP) pour contrer le brute-force.....	7
2	CONCLUSION.....	8

1 SPÉCIFICATIONS

1.1 Titre

Secured WebShop

1.2 Description

Le but de ce projet est de vous fournir une application web afin d'y implémenter un certain nombre de bonnes pratiques permettant de combler les failles de sécurité les plus courantes telles que listées dans le Top 10 2025 OWASP.

1.3 Matériel et logiciels à disposition

- Un ordinateur standard de la section informatique avec Docker Desktop
- Un dépôt GitHub https://github.com/ETML-INF/secured_webshop

1.4 Prérequis

Avoir fait le Module P_183 : Sécurité des applications

1.5 Liste des activités obligatoires

1.5.1 Implémenter une page de login en frontend

```
<ul>
  <label for="email">E-mail :</label>
  <input type="text" id="email" name="email" required />
  <label for="mdp">Mot de passe :</label>
  <input type="password" id="mdp" name="password" required />
</ul>
<button type="submit">Envoyer</button>
```

Tout d'abord j'ai commencé par créer le formulaire avec les champs où l'on peut remplir l'email, le mot de passe et le bouton envoyer pour envoyer les saisies de l'utilisateur

```
loginForm.addEventListener("submit", async (e) => {
  e.preventDefault();

  const email = document.getElementById("email").value;
  const password = document.getElementById("mdp").value;

  try {
    const response = await fetch("/api/auth/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ email, password }),
    });
  }
});
```

J'ai ensuite codé un script pour récupérer l'email et le mot de passe dès que l'utilisateur valide le formulaire. Pour éviter que la page ne s'actualise, j'utilise `e.preventDefault()`. Enfin, j'envoie ces informations au serveur avec `fetch` en les transformant en format JSON pour qu'elles soient traitées par la base de données.

1.5.2 Implémenter une page d'inscription en frontend

```
<form action="/api/auth/register" method="post">
  <ul>
    <li>
      <label for="username">Nom :</label>
      <input type="text" id="username" name="username" required />
    </li>
    <li>
      <label for="address">Adresse :</label>
      <input type="text" id="address" name="address" required />
    </li>
    <li>
      <label for="email">E-mail :</label>
      <input type="email" id="email" name="email" required />
    </li>
    <li>
      <label for="mdp">Mot de passe :</label>
      <input type="password" id="mdp" name="password" required />
      <div id="strength-container">
        <div id="strength-bar"></div>
        <span id="strength-text">Force du mot de passe</span>
      </div>
    </li>
  </ul>
  <button type="submit">Envoyer</button>
</form>
```

Pour la création de cette page, j'ai structuré un formulaire HTML qui pointe directement vers ma route API `/api/auth/register` en méthode `POST`. Chaque champ possède un attribut `name` précis pour que le backend puisse récupérer les données (via `req.body`) sans erreur.

1.5.3 Remplacer les mots de passes en clair dans la base par un hash

Pour sécuriser l'authentification, j'ai fait en sorte que les mots de passe ne soient jamais stockés en clair. Dès la réception du formulaire, je passe la valeur du champ `password` dans la fonction `bcrypt.hash()` pour générer une empreinte chiffrée unique que j'insère ensuite en base de données. De cette manière, même en cas de fuite, les données restent illisibles. Lors de

la connexion, il me suffit d'utiliser `bcrypt.compare()` pour vérifier que le mot de passe saisi correspond bien au hash stocké, garantissant ainsi une sécurité maximale sans jamais manipuler le mot de passe brut.

1.5.4 Ajouter un sel

Pour renforcer la sécurité, j'ai utilisé un système de sel généré automatiquement par `bcrypt.hash()`. L'idée est de rendre chaque hash unique : même si deux utilisateurs choisissent le même mot de passe, leurs empreintes en base de données seront totalement différentes. Comme ce sel est directement intégré dans le hash final, cela bloque les attaques par tables pré-calculées (rainbow tables) et empêche un attaquant de deviner des mots de passe identiques en comparant les colonnes. C'est une étape indispensable pour rendre chaque compte vraiment indépendant et plus dur à pirater.

1.5.5 Ajouter un poivre

```
const pwdPeper = password + process.env.PEPPER;
```

```
PEPPER=GZG4g78FUFü!EU%UIHIFU
```

J'ai créé le poivre dans le `.env` puis je l'ai ajouté au mot de passe avant le hash.

1.5.6 Corriger les requêtes existantes afin de prévenir l'injection SQL

```
const query = "SELECT * FROM users WHERE email = ?";
```

```
const updateQuery = "UPDATE users SET refresh_token = ? WHERE id = ?";
```

Pour prévenir l'injection sql j'ai mis les `= ?` qui permet de traiter l'entré en chaîne de caractère et non pas comme du code exécutable.

1.5.7 Implémenter l'utilisation d'un token JWT

Pour sécuriser l'API, j'ai implémenté la bibliothèque `jsonwebtoken` afin de gérer l'authentification de manière totalement décentralisée et éviter de stocker des sessions sur le serveur. Au moment du `login()`, une fois le mot de passe validé, le contrôleur construit un payload contenant l'id, l'email et le rôle du user, puis génère le jeton via `jwt.sign()`. Ce token est signé avec une clé secrète stockée dans `process.env.JWT_SECRET`, ce qui garantit que personne ne peut falsifier les données côté client. Pour sécuriser les accès, le middleware `verifyToken()` intercepte les requêtes sur les routes protégées (comme sur le profil ou le panel admin), extrait le jeton du header `Authorization: Bearer` et le valide avec `jwt.verify()`. Si la signature est bonne, le contenu décodé est directement injecté dans `req.user`, ce qui permet aux contrôleurs suivants de savoir instantanément quel utilisateur fait la requête et d'agir en conséquence.

1.5.8 Ajouter les rôles administrateur et utilisateur dans le JWT et protéger les routes d'administration

Pour gérer les accès, j'ai intégré directement le rôle du user dans le payload du JWT (id, email, role) dès le login(), ce qui évite de retaper dans la base de données après.

La protection des routes sensibles se fait en combinant deux middlewares dans auth.js :

1. verifyToken() : Valide le jeton et injecte son contenu dans req.user.
2. isAdmin() : Vérifie que req.user.role vaut bien "admin" (passé en minuscules pour la sécurité).

Si l'un des deux verrous saute, la route (comme GET /users) renvoie un accès refusé. En gros, il suffit de mettre un user en "admin" dans la base de données pour que son prochain JWT lui ouvre automatiquement les droits.

1.6 Liste des activités « faciles » à choix

1.6.1 Mettre en place le HTTPS

J'ai intégré HTTPS directement dans server.js via les modules natifs https et fs pour chiffrer les échanges en local. Le serveur charge les fichiers de certificat (localhost+1-key.pem et localhost+1.pem) avec fs.readFileSync(), puis encapsule l'application Express dans https.createServer(). Du coup, l'API écoute sur https://localhost:8080 et toutes les requêtes sont sécurisées. C'est une config de dev locale (le certificat doit être présent à la racine pour que ça tourne), et si on veut forcer la redirection HTTP vers HTTPS plus tard, il faudra juste lancer un petit serveur HTTP en parallèle pour rediriger les flux.

1.6.2 Mettre en place une politique de mot de passe fort

J'ai intégré un script de validation en temps réel. Ce dernier analyse la complexité du mot de passe (longueur, chiffres, caractères spéciaux) et met à jour dynamiquement une barre de force colorée. Cela permet de guider l'utilisateur pour qu'il choisisse un compte sécurisé avant même de valider le formulaire.

1.6.3 Limiter la durée du token JWT actuel et implémenter un refresh token pour rester connecté sur une longue période

Pour blinder la sécurité, j'ai séparé l'authentification en deux jetons distincts dès le login() : un Access Token éphémère (ex: 10 secondes via expiresIn) pour valider les requêtes courantes, et un Refresh Token persistant (7 jours) stocké directement en base de données. Dès que l'Access Token expire, le front appelle l'endpoint /api/auth/refresh. Le serveur vérifie la présence du Refresh Token en base via un SELECT, valide sa signature cryptographique, et renvoie un jeton d'accès tout neuf.

1.6.4 Effectuer un audit des dépendances NPM

```
$ npm audit fix  
  
added 9 packages, removed 5 packages, changed 27 packages, and audited 151 packages in 3s  
  
22 packages are looking for funding  
  run `npm fund` for details  
  
found 0 vulnerabilities
```

1. Modifications des paquets

L'installation a été mise à jour de la manière suivante :

- **Ajouts** : 9 nouveaux paquets ont été installés.
- **Suppressions** : 5 paquets ont été retirés du projet.
- **Changements** : 27 paquets existants ont été mis à jour ou modifiés pour résoudre des problèmes de sécurité.
- **Portée** : Au total, 151 paquets ont été inspectés durant ce processus qui a duré seulement 3 secondes.
- **Financement** : 22 paquets qui demandent un financement pour avoir plus de détails il faut faire npm fund
- **Vulnérabilité** : 0 vulnérabilité on été trouver après l'audit fix

1.6.5 Gérer les exceptions afin de ne pas retourner trop d'information en cas d'erreur

Pour éviter les fuites d'infos en cas de bug, j'ai séparé complètement les logs du serveur et les réponses de l'API. Au lieu d'afficher une grosse erreur technique, l'API renvoie des messages tout simples et flous comme "Erreur serveur" ou "Identifiants incorrects". Ça permet de cacher la structure de la base et d'empêcher les hackers de tester des emails.

1.7 Liste des activités « moyennes » à choix

1.7.1 Limiter le nombre de tentatives de login (5 essais par minute par IP) pour contrer le brute-force

Pour bloquer les attaques par brute-force, j'ai utilisé le package express-rate-limit appliqué uniquement sur les routes /api/auth. Le système est configuré pour limiter chaque adresse IP à un maximum de 5 requêtes par minute. Dès la 6ème tentative, le middleware bloque la demande avant même qu'elle n'atteigne le contrôleur ou la base de données, puis renvoie un code HTTP 429 avec le message "Trop de tentatives. Réessayez dans une minute".

2 CONCLUSION

Pour ma conclusion j'ai trouvé ce projet intéressant car la sécurité est un domaine qui m'intéresse mais j'ai trouvé qu'utiliser express était assez embêtant car on ne l'avait jamais utilisé au paravent et que je n'aime pas beaucoup les Framework en général, mais en global j'ai quand même apprécié ce cours.